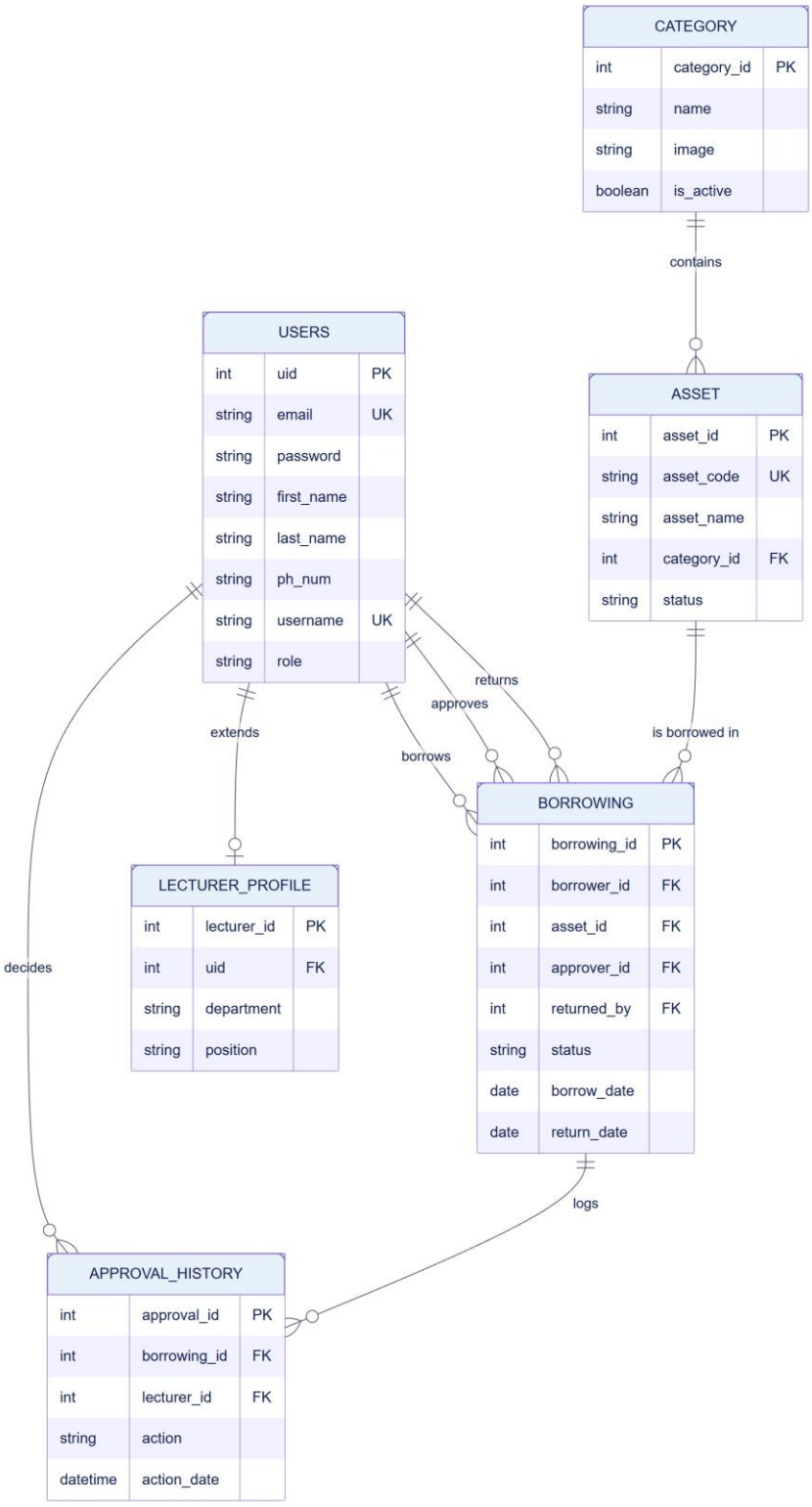
Assets Borrowing System

Running on a real Android emulator

This document is the mobile equivalent of the earlier web walkthrough. Every screenshot on the following pages was captured directly off a virtual Android phone (Google Pixel 5 AVD, Android 14) with **adb**, while the installed `.apk` talked to a live backend (Node.js + MySQL) over the network — not a mock, not the web build.

Codebase	asset_borrowing_system (progress3 branch) + Node/MySQL backend
Package	com.example.asset_borrowing_system
Toolchain	Flutter 3.47.4 · Android SDK 36 · Gradle 8.14
Emulator	pixel_test AVD · Android 14 (API 34) system image
Backend	Express + MySQL 8, running in Docker, verified via curl and live login

The application supports three user roles — Student, Lecturer, and Staff — all sharing a single mobile front-end built-in Flutter and communicating with an Express and MySQL 8 backend running in Docker. The system provides authentication, asset browsing, borrowing requests, approval, returns, and transaction history, with all data stored in a relational MySQL database. The diagram below shows the six core tables and how they relate to one another.



Foreign keys tie each borrowing record to its borrower, the requested asset, the approving lecturer, and the staff member who processed the return, so the complete audit trail for one transaction lives in a single row without duplicating user data. The category and asset tables are linked in the same way, so every physical unit belongs to exactly one of the four asset categories.

Cold Start

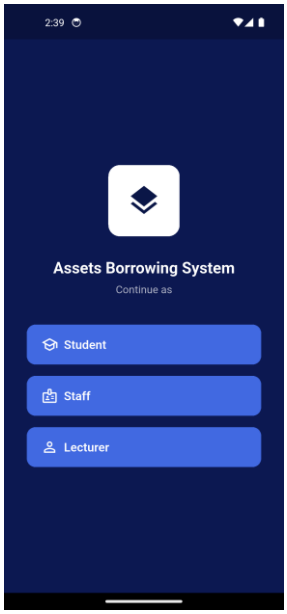
the app's own launch sequence, not a browser tab



STEP 00

App launch splash

The OS's own cold-start splash for the installed package — this only ever appears for a real installed app, confirming this is not a wrapped web page running inside a browser shell.



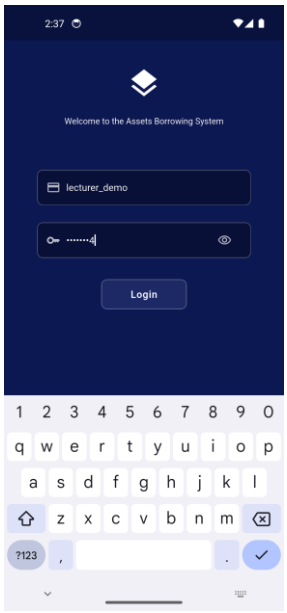
STEP 01

Role select

The landing screen added this session. Previously there was no way to switch between Student, Staff and Lecturer without editing `main.dart`'s `initialRoute` and rebuilding the app.

Lecturer

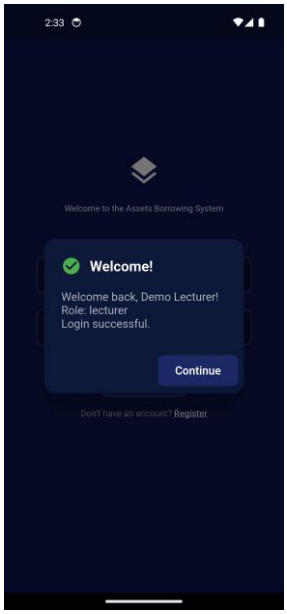
lib/lecturer/.dart*



STEP 02

Sign in

Credentials for `lecturer_demo`, a real row seeded into the MySQL database this session specifically to exercise this role.

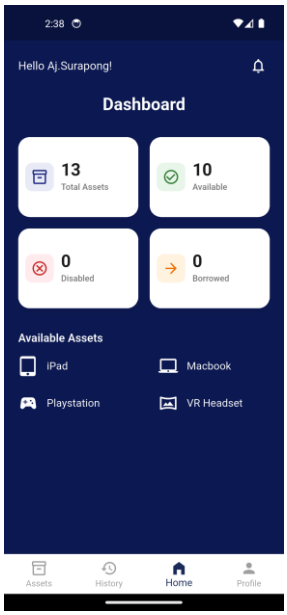


STEP 03

Real authentication

“Welcome back, Demo Lecturer! Role: lecturer.” This text is returned directly by `/api/login` after verifying the submitted password against an argon2 hash stored in the database — there is no local/offline fallback for this check.

✓ VERIFIED AGAINST LIVE MYSQL

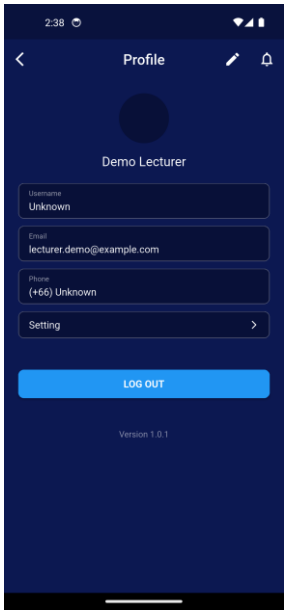


STEP 04

Dashboard

`13 Total Assets, 10 Available` — pulled live from a brand-new `/api/lecturer/dashboard` endpoint built this session. The Lecturer screens originally called API methods that didn't exist anywhere in the codebase; this is one of several gaps filled in to get the app running end-to-end.

✓ VERIFIED AGAINST LIVE MYSQL



STEP 05

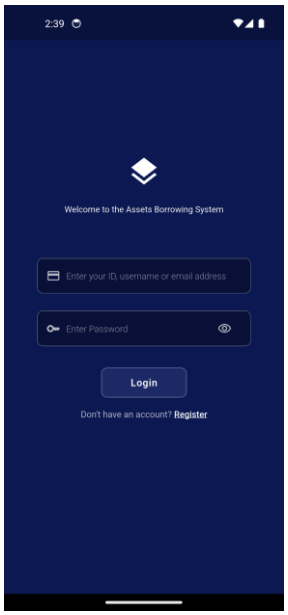
Profile

Name and email loaded from `/api/lecturer/profile/:uid`, keyed off the session saved at login — another gap fixed this session (the lecturer login previously never saved a session at all, so this screen could not have worked before).

✓ VERIFIED AGAINST LIVE MYSQL

Student

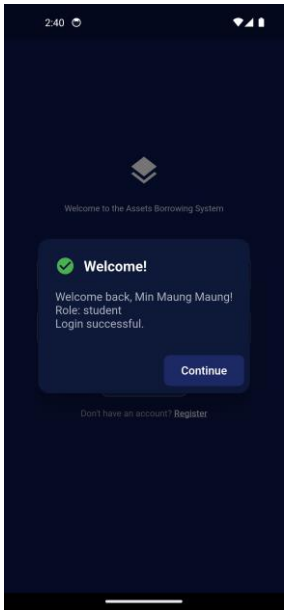
lib/student/.dart*



STEP 06

Sign in

A distinct screen from Lecturer's — note the `Register` link, which only the Student flow has.



STEP 07

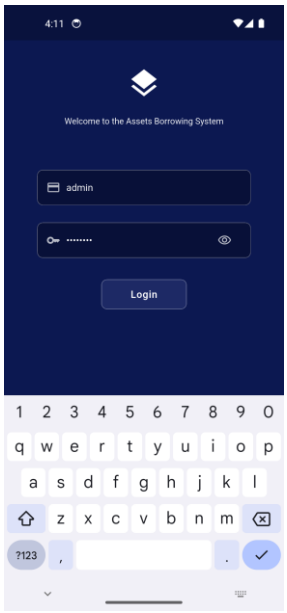
Real authentication

Logged in as `minmaung21`, an existing seeded student account. “Welcome back, Min Maung Maung! Role: student.” confirms the name came from the database row, not a placeholder string.

✓ VERIFIED AGAINST LIVE MYSQL

Staff

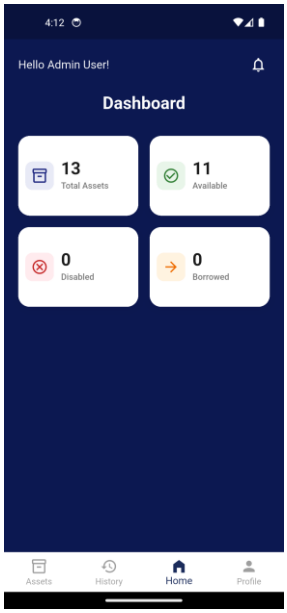
lib/staff/.dart — built from scratch this session*



STEP 08

Sign in

Credentials for `admin`. The `users.role` column previously had no `staff` value at all — every branch's schema only supported `student` and `lecturer/admin`. Migrated the enum and this account to `staff` this session.

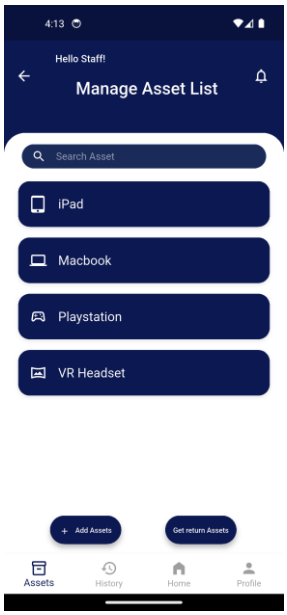


STEP 09

Dashboard

“Hello Admin User!” and `13 Total Assets / 11 Available` — both pulled live via a new `/api/staff/dashboard` endpoint. This screen previously showed hardcoded numbers (50/30/5/15) with no backend call at all.

✓ VERIFIED AGAINST LIVE MYSQL

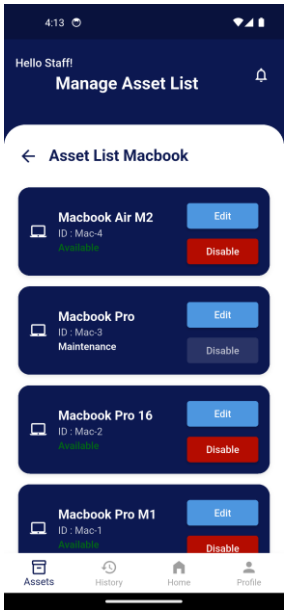


STEP 10

Manage Asset List

Categories (iPad, Macbook, Playstation, VR Headset) loaded from `/api/student/categories` — reused directly since browsing is identical across roles.

✓ VERIFIED AGAINST LIVE MYSQL

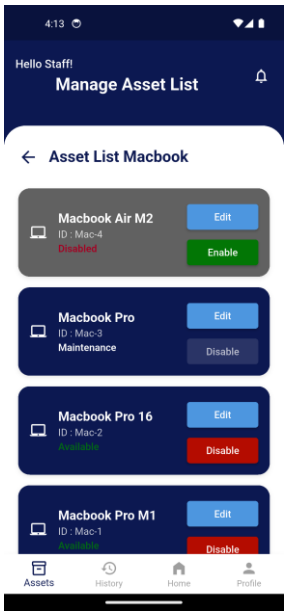


STEP 11

Per-unit status

A new `/api/staff/assets/category/:id` endpoint returns every unit *regardless of status* (unlike the student endpoint, which only shows `available`). Note *Macbook Pro — Maintenance*: its Disable button is correctly greyed out, since the spec only allows disabling available assets.

✓ VERIFIED AGAINST LIVE MYSQL

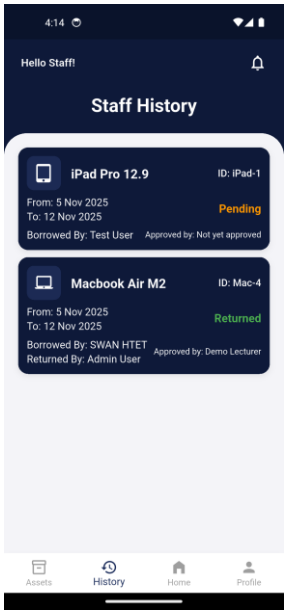


STEP 12

Live Disable action

Tapped `Disable` on *Macbook Air M2* — the card greys out, the status flips to Disabled, and the button becomes Enable, all from a genuine `PUT /api/staff/assets/:id/disable` round trip, not local state.

✓ VERIFIED AGAINST LIVE MYSQL

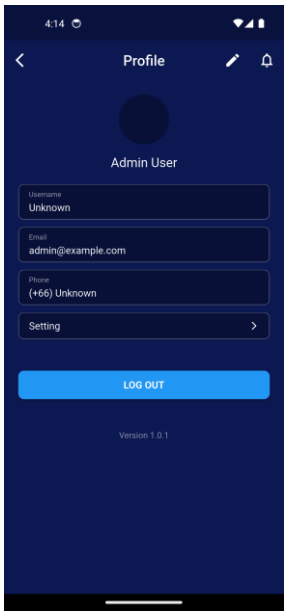


STEP 13

History — every user, not just their own

Staff is the only role that sees *everyone's* activity. This card shows a complete real chain: *Macbook Air M2* borrowed by SWAN HTET, approved by Demo Lecturer, and returned by Admin User — three different accounts, one real transaction, assembled by a new `/api/staff/history` query joining borrower, approver and returner.

✓ VERIFIED AGAINST LIVE MYSQL



STEP 14

Profile

“Admin User” / “admin@example.com” loaded from `/api/staff/profile/:uid`, keyed off the session this login now actually saves (it previously didn't call any backend at all).

✓ VERIFIED AGAINST LIVE MYSQL

Captured by installing the built `app-debug.apk` on an AVD (`pixel_test`, Android 34 image) and driving it with `adb shell input` + `adb exec-out screencap` — the same phone anyone running this project locally would see. No manual interaction with the emulator window was required to produce these screenshots. A screen recording of the Staff flow (Assets → live Disable/Enable → History → Profile) was captured the same way with `adb shell screenrecord` and is delivered alongside this PDF.